Lab 1: Python Warm-up – Getting Your Feet Wet (with VS Code Dev Containers)

Objective

By the end of this lab, you will be able to:

1. Set up and use a Python 3 Dev Container in VS Code.
2. Create and run simple Python scripts within the Dev Container.
3. Declare and use variables of different basic types (strings, integers, floats).
4. Perform basic input from the console and output to the console.
5. Write simple conditional statements.
6. Use the `requests` library to make a basic HTTP GET request to an external service (`wttr.in`) and display its output.
7. (Bonus) Modify your script to fetch weather for a user-specified city.

Prerequisites:

- Visual Studio Code installed.
- Docker Desktop (or another Docker provider compatible with VS Code) installed and running.
- The "Dev Containers" extension installed in VS Code (ID: `ms-vscode-remote.remote-containers`).
- Internet access.

Instructions:

Part 0: Setup Your Dev Container & First Python Script

1. Create a Project Folder:
   - On your local machine (outside of any container yet), create a new folder for this course's projects, e.g., `IoT_System_Integration_Course`.
   - Inside that, create a folder for today's lab: `Day2_Python_Labs`.
2. Open Project Folder in VS Code:
   - Open VS Code.
   - Go to `File > Open Folder...` and select the `Day2_Python_Labs` folder you just created.
3. Initialize Dev Container:
   - Once the folder is open in VS Code, open the Command Palette:

    - `View > Command Palette...` or `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS).
  - In the Command Palette, type `Dev Containers: Add Dev Container Configuration Files...` and select it.
  - You'll be prompted to "Select a Dev Container Configuration Template". Type `Python 3` in the search box and select the "Python 3" definition (it should be one of the first options, provided by Microsoft).
  - VS Code will create a `.devcontainer` folder in your project with a `devcontainer.json` file (and possibly a `Dockerfile`).
4. Reopen in Container:
  - A notification will appear at the bottom right of VS Code: "Folder contains a Dev Container configuration file. Reopen in Container to use it." Click the "Reopen in Container" button.
  - VS Code will now build and start your Dev Container. This might take a few minutes the first time as it downloads the Python Docker image. You can see the progress in the terminal output.
  - Once it's ready, the VS Code window will reload, and the status bar at the bottom-left will indicate you are connected to "Dev Container: Python 3".
5. Create Your First Script File:
  - In the VS Code Explorer (left sidebar), right-click inside the `Day2_Python_Labs` area (or on the folder name itself) and select "New File...".
  - Name the file `weather_app.py`.
6. "Hello, Python!"
  - Open `weather_app.py` by clicking on it.
  - Type the following line of code:

```
print("Hello, Python Weather App from inside the Dev Container!")
```

7. Running the Script:
  - Open the integrated terminal in VS Code: `Terminal > New Terminal`. This terminal is *inside* your Dev Container.
  - In the terminal, you should already be in your project directory (`/workspaces/Day2_Python_Labs` or similar).
  - Run your script by typing:

```
python weather_app.py
```

    (The Dev Container is configured so `python` usually points to the correct Python 3 version).
  - You should see "Hello, Python Weather App from inside the Dev Container!" printed. Congratulations!

Part 1: Variables and Basic Data Types

In this part, you'll experiment with how Python handles variables.

1. Strings:
   - Add the following lines to your `weather_app.py` script (you can comment out or delete the "Hello" line if you wish):

```
city_name = "Stockholm"
greeting_message = "Weather forecast for:"
print(greeting_message, city_name)
```

   - Run your script from the integrated VS Code terminal.
   - Experiment:
     - Try changing `city_name` to a different city.
     - Try concatenating strings using the `+` operator: `full_message = greeting_message + " " + city_name`. Print `full_message`.
2. Integers and Floats:
   - Add these lines:

```
current_temperature_celsius = 7 # An integer
expected_high_celsius = 12.5   # A float (decimal number)

print("Current temperature:", current_temperature_celsius, "C")
print("Expected high:", expected_high_celsius, "C")

# Basic arithmetic
temperature_difference = expected_high_celsius - current_temperature_celsius
print("Temperature rise expected:", temperature_difference, "C")
```

   - Run your script.
   - Experiment:
     - What happens if you try to add an integer and a float? What is the type of the result? You can check the type of a variable using `print(type(your_variable_name))`.
     - Example: `print(type(temperature_difference))`
3. Type Conversion (Casting):
   - Sometimes you need to convert between types.

```
sensor_id_str = "Sensor_00"
sensor_number = 1

# Let's try to combine them (this will cause an error initially)
# full_sensor_id = sensor_id_str + sensor_number
```

```
# print(full_sensor_id)

# Correct way: convert the integer to a string
full_sensor_id_corrected = sensor_id_str + str(sensor_number)
print("Full sensor ID:", full_sensor_id_corrected)

reading_str = "23.7"
reading_float = float(reading_str)
print("Sensor reading as float:", reading_float, type(reading_float))
```

- Uncomment the line that causes an error, run it to see the `TypeError`, then comment it out again and run with the corrected version.

Part 2: Console Input and Output

Now, let's make your script interactive.

1. Getting User Input:
   - The `input()` function is used to get text input from the user.
   - Add these lines:

```
user_name = input("What is your name? ")
print("Hello,", user_name + "!")

favorite_season = input("What is your favorite season for weather? ")
print(favorite_season, "is a great choice, " + user_name + "!")
```

   - Run the script. The VS Code terminal will pause and wait for you to type something and press Enter.
2. Formatted Output (f-strings):
   - Python's f-strings (formatted string literals) are a very convenient way to embed expressions inside string literals.
   - Modify your previous print statements or add new ones using f-strings:

```
# Using the variables from Part 1
print(f"Recap: Weather for {city_name}.")
print(f"{user_name}, the current temperature is
{current_temperature_celsius}C and the expected high is
{expected_high_celsius}C.")
```

   - Run and observe the output.

Part 3: Making a Web Request to `wttr.in`

This is where we connect Python to the outside world!

1. Install the `requests` Library (inside the Dev Container):
    - The Python 3 Dev Container usually comes with `pip` (Python's package installer) pre-installed.
    - In the VS Code integrated terminal (which is inside your Dev Container), type:

```
pip install requests
```

    - You should see messages indicating that `requests` (and its dependencies) are being downloaded and installed *inside the container*. This installation is isolated to your Dev Container and won't affect your main system's Python installation.
2. Import the Library:
    - At the very top of your `weather_app.py` script, add this line:

```
import requests
```

3. Fetch Weather for Stockholm:
    - Add the following code to your script:

```
# Define the base URL for wttr.in and the city
base_url = "http://wttr.in/"
target_city = "Stockholm"

# Construct the full URL. For wttr.in, you can just append the city name.
# We also add "?format=3" to get a concise, single-line forecast.
full_url = f"{base_url}{target_city}?format=3"

print(f"\nFetching weather for {target_city} from {full_url}...")

try:
    # Make the HTTP GET request
    response = requests.get(full_url)

    # Check if the request was successful (status code 200)
    if response.status_code == 200:
        # Print the weather forecast (which is the text content of the
response)
        print("Weather Report:")
        print(response.text)
    else:
        print(f"Error fetching weather: Status code {response.status_code}")
        print(f"Response text: {response.text}")

except requests.exceptions.RequestException as e:
    # Handle potential network errors (e.g., no internet connection from the
```

```
container)
    print(f"An error occurred during the request: {e}")
```

4. Run Your Script:
    - Save `weather_app.py` and run it from the integrated VS Code terminal.
    - You should see it attempt to fetch the weather and then print the forecast for Stockholm!

Part 4: Bonus Assignments

1. User-Specified City via Input:
    - Modify your script so that instead of hardcoding `target_city = "Stockholm"`, it asks the user for a city name using the `input()` function.
    - Use the user's input to construct the `full_url`.
    - Hint:

```
# ... (import requests and define base_url)
user_city = input("Enter the city you want the weather for: ")
full_url = f"{base_url}{user_city}?format=3"
# ... (the rest of your try-except block for requests.get)
```

2. (Advanced Bonus) User-Specified City via Command-Line Argument:
    - You'll need to import the `sys` module: `import sys` at the top of your script.
    - Command-line arguments are available in the `sys.argv` list.
    - Hint:

```
import sys
import requests

# ... (define base_url)

if len(sys.argv) > 1:
    user_city_arg = sys.argv[1]
else:
    user_city_arg = input("No city provided as argument. Enter city: ")

full_url = f"{base_url}{user_city_arg}?format=3"
# ... (the rest of your try-except block for requests.get)
```

    - To run this from the VS Code integrated terminal:

```
python weather_app.py London
```

    or

```
python weather_app.py "New York"
```

Lab Completion:

- Ensure your script can successfully fetch and display the weather for Stockholm from within the Dev Container.
- If you attempt the bonus, ensure it works as described.
- Be prepared to briefly show your working script or discuss any challenges.

Troubleshooting Tips (within Dev Container):

- `ModuleNotFoundError: No module named 'requests'`: Make sure you ran `pip install requests` *inside the Dev Container's terminal*. If you ran it outside, the container won't see it.
- Typos/Indentation Errors: Same as before, Python is strict. VS Code's Python extension should help catch some of these.
- Network Issues from Container: Ensure your Docker setup allows containers to access the internet. Usually, this is default. If `wttr.in` is down, the request will fail. The `try...except` block should handle this.
- Dev Container Not Starting: Check Docker Desktop is running. Look at the logs in the VS Code "OUTPUT" tab (select "Dev Containers" from the dropdown) for error messages.